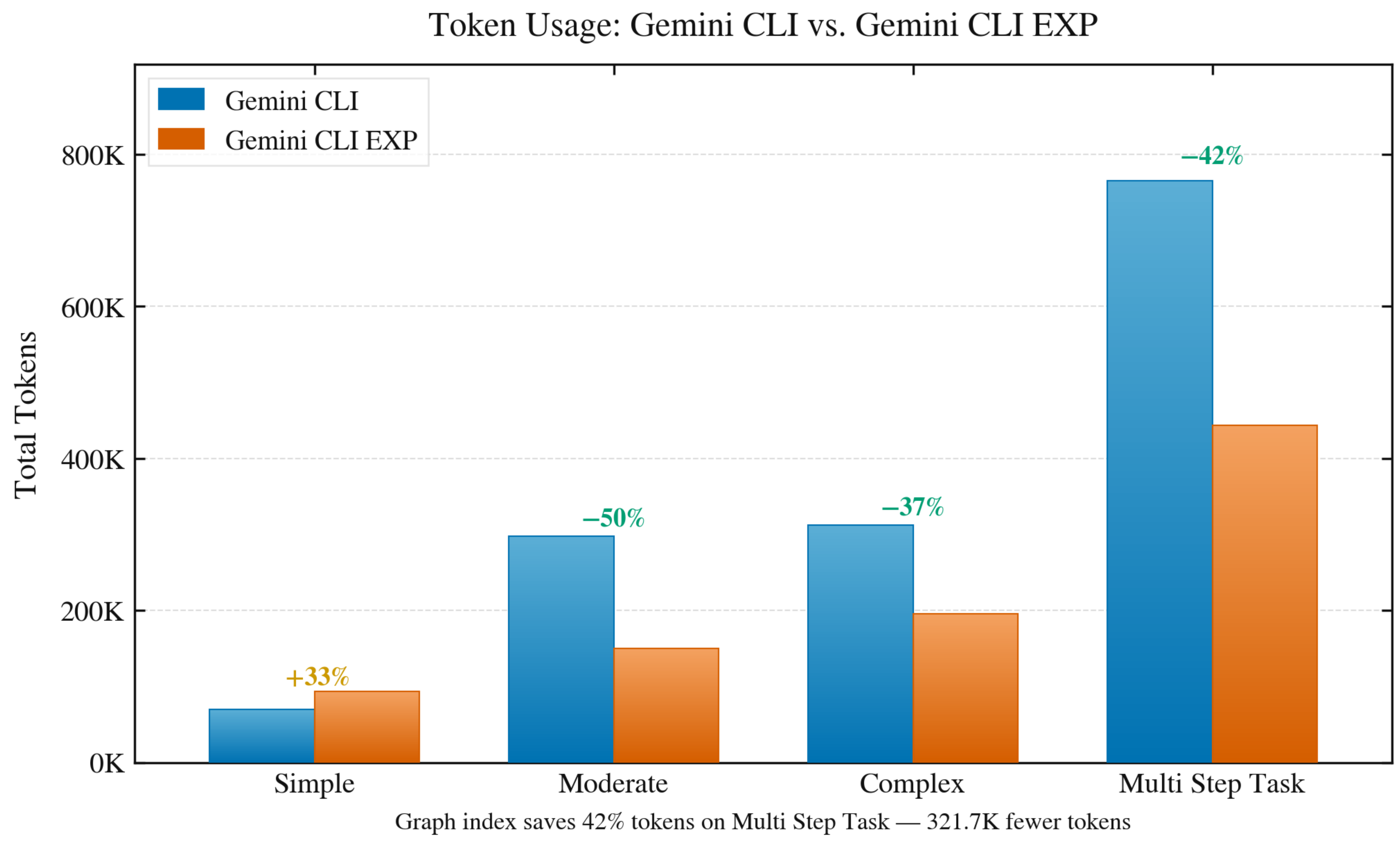
TRY GEMINI CLI EXPERIMENTAL FORK



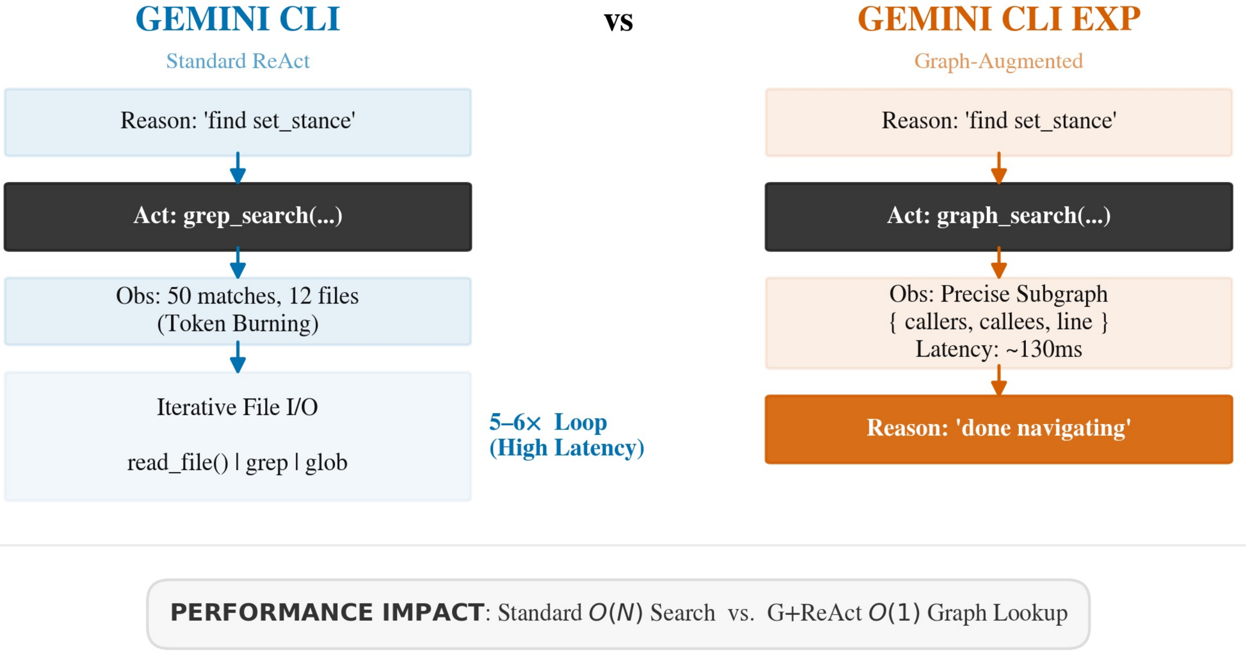
1. Graph index replaces file scanning in agent loops.
2. Single graph call resolves any symbol in 38ms.
3. 37–50% fewer tokens on moderate to complex tasks.
4. 43–50% fewer model requests, same quality output.
5. Up to 46% faster wall time on complex tasks.
6. For every task main & sub agent = share's the graph.
7. Index built once, auto-refreshed every session start.
8. O(1) symbol lookup replaces O(n) grep-read loop.

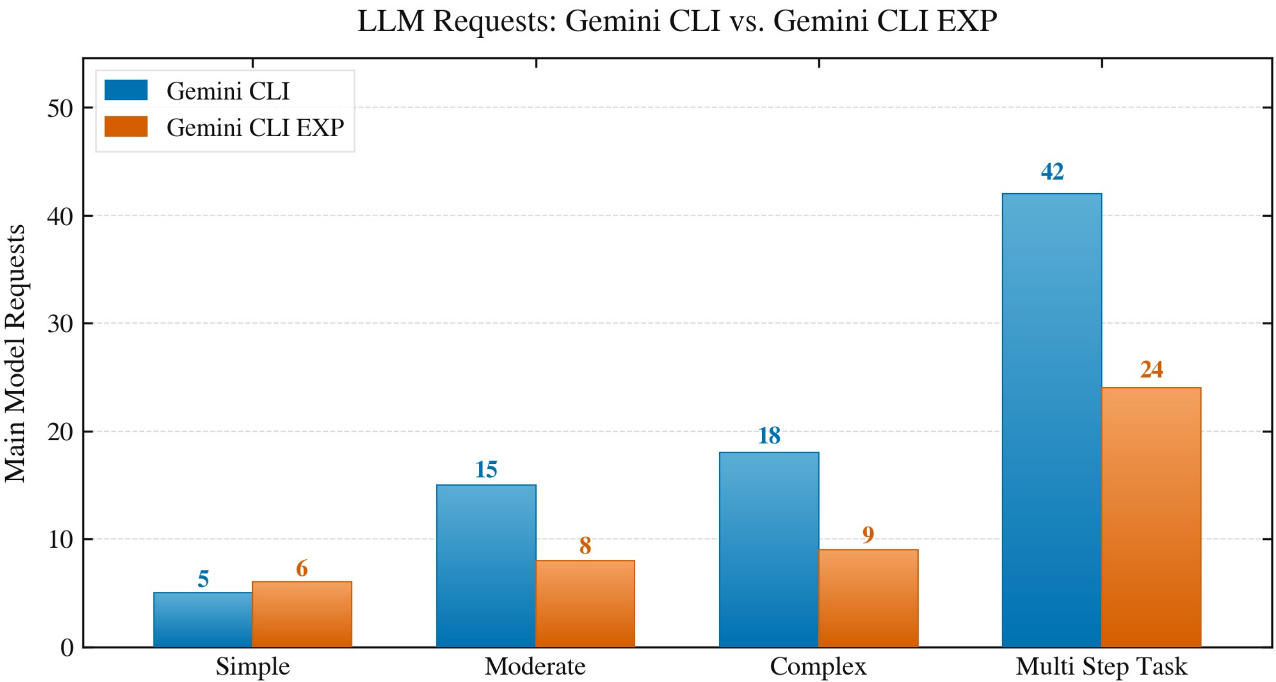
* Building more features!

Ashwin Shirke

Repo : git@github.com:Ashwin3919/gemini-cli-exp.git (branch : graph-idx)

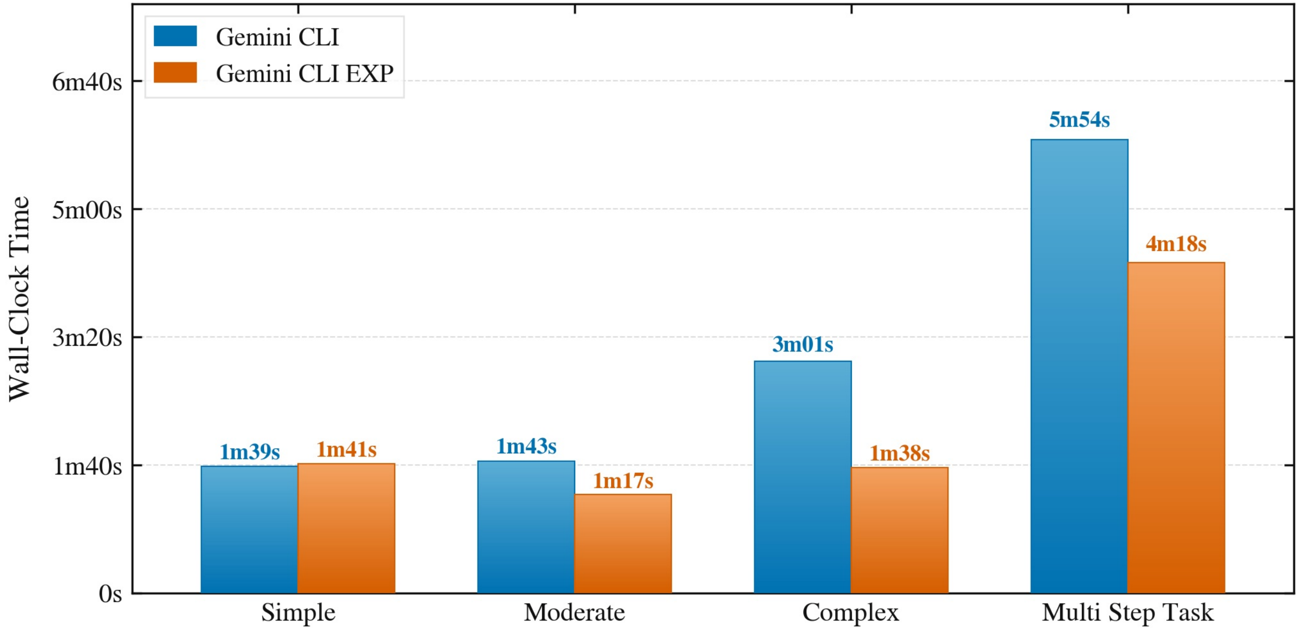
Efficiency plots ->


GEMINI CLI
Standard ReAct
vs
GEMINI CLI EXP
Graph-Augmented
Reason: 'find set_stance'
Act: grep_search(...)
Obs: 50 matches, 12 files
(Token Burning)
Iterative File I/O
read_file() | grep | glob
5–6× Loop
(High Latency)
Reason: 'find set_stance'
Act: graph_search(...)
Obs: Precise Subgraph
{ callers, callees, line }
Latency: ~130ms
Reason: 'done navigating'
PERFORMANCE IMPACT: Standard $O(N)$ Search vs. G+ReAct $O(1)$ Graph Lookup

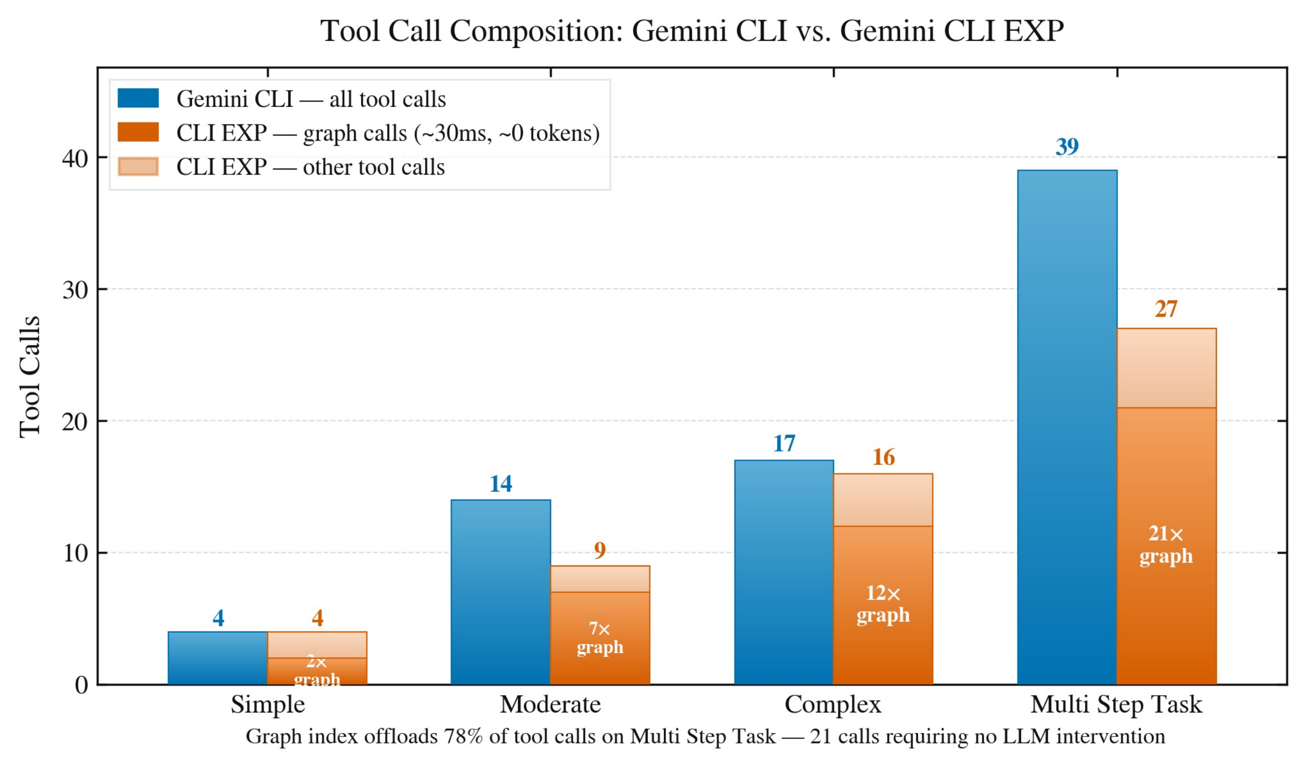


Graph index saves 43% requests on Multi Step Task — 18 fewer LLM round-trips

Session Duration: Gemini CLI vs. Gemini CLI EXP



Graph index saves 27% time on Multi Step Task — 1m36s of developer wait time


Tool Call Composition: Gemini CLI vs. Gemini CLI EXP
Gemini CLI — all tool calls
CLI EXP — graph calls (~30ms, ~0 tokens)
CLI EXP — other tool calls
Tool Calls
0
10
20
30
40
4
4
2×
graph
14
9
7×
graph
17
16
12×
graph
39
27
21×
graph
Simple
Moderate
Complex
Multi Step Task
Graph index offloads 78% of tool calls on Multi Step Task — 21 calls requiring no LLM intervention

Thank you!

If you liked it drop a * to repo : https://github.com/Ashwin3919/gemini-cli-exp.git

Ashwin Shirke